

LAB ASSIGNMENT – 11.5

NAME – SHREE PRIYA

2303A51465

BATCH – 29

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Generate a simple Python class for Stack with push, pop, peek, and is_empty methods including basic docstrings

CODE:

```
class Stack:
```

```
    """Simple Stack implementation using a list."""
```

```
    def __init__(self):
```

```
        """Initialize an empty stack."""
```

```
        self.items = []
```

```
    def push(self, item):
```

```
        """Add an item to the top of the stack."""
```

```
        self.items.append(item)
```

```
    def pop(self):
```

```
        """Remove and return the top item of the stack."""
```

```
        if not self.is_empty():
```

```
            return self.items.pop()
```

```
        return "Stack is empty"
```

```
def peek(self):
    """Return the top item without removing it."""
    if not self.is_empty():
        return self.items[-1]
    return "Stack is empty"

def is_empty(self):
    """Check if the stack is empty."""
    return len(self.items) == 0

# Example usage:

if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    print(stack.peek()) # Output: 2
    print(stack.pop()) # Output: 2
    print(stack.is_empty()) # Output: False
    print(stack.pop()) # Output: 1
    print(stack.is_empty()) # Output: True
```

```
C: > Users > Shree priya > Downloads > aac11.5.py > Stack > pop
2 class Stack:
3     """Simple Stack implementation using a list."""
4
5     def __init__(self):
6         """Initialize an empty stack."""
7         self.items = []
8
9     def push(self, item):
10        """Add an item to the top of the stack."""
11        self.items.append(item)
12
13    def pop(self):
14        """Remove and return the top item of the stack."""
15        if not self.is_empty():
16            return self.items.pop()
17        return "Stack is empty"
18
19    def peek(self):
20        """Return the top item without removing it."""
21        if not self.is_empty():
22            return self.items[-1]
23        return "Stack is empty"
24
25    def is_empty(self):
26        """Check if the stack is empty."""
27        return len(self.items) == 0
28
29    # Example usage:
30    if __name__ == "__main__":
31        stack = Stack()
32        stack.push(1)
33        stack.push(2)
34        print(stack.peek()) # Output: 2
35        print(stack.pop())  # Output: 2
36        print(stack.is_empty()) # Output: False
37        print(stack.pop())  # Output: 1
38        print(stack.is_empty()) # Output: True
```

PROBLEMS OUTPUT TERMINAL PORTS

> TERMINAL

```
18.0-win32-x64\bundled\libs\debugpy\launcher' '58703' '--' 'C:\Users\Shree priya\Downloads\aac11.5.py'
2
2
False
1
True
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Generate a simple Python Queue class using lists with enqueue, dequeue, peek, and size methods including basic docstrings.

CODE:

class Queue:

"""Simple Queue implementation using a list."""

def __init__(self):

"""Initialize an empty queue."""

self.items = []

def enqueue(self, item):

"""Add an element to the queue."""

self.items.append(item)

def dequeue(self):

"""Remove and return the first element from the queue."""

if not self.is_empty():

return self.items.pop(0)

return "Queue is empty"

def peek(self):

"""Return the front element without removing it."""

if not self.is_empty():

return self.items[0]

return "Queue is empty"

def size(self):

"""Return the number of elements in the queue."""

return len(self.items)

def is_empty(self):

"""Check if the queue is empty."""

```
    return len(self.items) == 0
```

Example usage:

```
if __name__ == "__main__":
```

```
    queue = Queue()
```

```
    queue.enqueue(1)
```

```
    queue.enqueue(2)
```

```
    print(queue.peek()) # Output: 1
```

```
    print(queue.dequeue()) # Output: 1
```

```
    print(queue.size()) # Output: 1
```

```
    print(queue.is_empty()) # Output: False
```

```
    print(queue.dequeue()) # Output: 2
```

```
    print(queue.is_empty()) # Output: True
```

```
C:\Users\Shree priya\Downloads> aac115.py Queue is_empty
5 class Queue:
6     def __init__(self):
7         """Initialize an empty queue."""
8         self.items = []
9
10    def enqueue(self, item):
11        """Add an element to the queue."""
12        self.items.append(item)
13
14    def dequeue(self):
15        """Remove and return the first element from the queue."""
16        if not self.is_empty():
17            return self.items.pop(0)
18        return "Queue is empty"
19
20    def peek(self):
21        """Return the front element without removing it."""
22        if not self.is_empty():
23            return self.items[0]
24        return "Queue is empty"
25
26    def size(self):
27        """Return the number of elements in the queue."""
28        return len(self.items)
29
30    def is_empty(self):
31        """Check if the queue is empty."""
32        return len(self.items) == 0
33
34    # Example usage:
35    if __name__ == "__main__":
36        queue = Queue()
37        queue.enqueue(1)
38        queue.enqueue(2)
39        print(queue.peek()) # Output: 1
40        print(queue.dequeue()) # Output: 1
41        print(queue.size()) # Output: 1
42        print(queue.is_empty()) # Output: False
43        print(queue.dequeue()) # Output: 2
44        print(queue.is_empty()) # Output: True
45
PROBLEMS OUTPUT TERMINAL PORTS
> TERMINAL
1
1
1
False
2
True
PS C:\Users\Shree priya\Downloads>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

Generate a simple Python implementation of a Singly Linked List with Node and LinkedList classes including insert and display methods with basic docstrings.

class Node:

"""Node class for storing data and next reference."""

def __init__(self, data):

"""Initialize node with data."""

self.data = data

self.next = None

class LinkedList:

"""Singly Linked List implementation."""

def __init__(self):

"""Initialize empty linked list."""

self.head = None

def insert(self, data):

"""Insert a new node at the end of the list."""

new_node = Node(data)

if self.head is None:

self.head = new_node

return

temp = self.head

while temp.next:

temp = temp.next

temp.next = new_node

```
def display(self):
    """Display all elements in the linked list."""
    temp = self.head
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")

# Example usage
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)

    print("Linked List contents:")
    linked_list.display()
```



```

28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Generate a simple Python HashTable class using chaining for collision handling with insert, search, and delete methods and basic docstrings.

```
class HashTable:
```

```
    """Simple Hash Table implementation using chaining."""
```

```
    def __init__(self, size=10):
```

```
        """Initialize hash table with given size."""
```

```
        self.size = size
```

```

self.table = [[] for _ in range(size)]

def hash_function(self, key):
    """Compute index for a key."""
    return hash(key) % self.size

def insert(self, key, value):
    """Insert key-value pair into the hash table."""
    index = self.hash_function(key)
    self.table[index].append((key, value))

def search(self, key):
    """Search for a key and return its value."""
    index = self.hash_function(key)
    for k, v in self.table[index]:
        if k == key:
            return v
    return "Key not found"

def delete(self, key):
    """Delete a key-value pair from the hash table."""
    index = self.hash_function(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            del self.table[index][i]
            return "Deleted"
    return "Key not found"

# Example usage
if __name__ == "__main__":
    ht = HashTable()
    ht.insert("name", "Faaaaaaaaaah")

```

```

ht.insert("age", 30)

print(ht.search("name")) # Output: Alice

print(ht.search("age")) # Output: 30

print(ht.delete("name")) # Output: Deleted

print(ht.search("name")) # Output: Key not found

print(ht.delete("name")) # Output: Key not found

```

```

1 class HashTable:
2     def __init__(self, size=10):
3         """Initialize hash table with given size."""
4         self.size = size
5         self.table = [[] for _ in range(size)]
6
7     def hash_function(self, key):
8         """Compute index for a key."""
9         return hash(key) % self.size
10
11    def insert(self, key, value):
12        """Insert key-value pair into the hash table."""
13        index = self.hash_function(key)
14        self.table[index].append((key, value))
15
16    def search(self, key):
17        """Search for a key and return its value."""
18        index = self.hash_function(key)
19        for k, v in self.table[index]:
20            if k == key:
21                return v
22        return "Key not found"
23
24    def delete(self, key):
25        """Delete a key-value pair from the hash table."""
26        index = self.hash_function(key)
27        for i, (k, v) in enumerate(self.table[index]):
28            if k == key:
29                del self.table[index][i]
30                return "Deleted"
31        return "Key not found"
32
33    # Example usage
34    if __name__ == "__main__":
35        ht = HashTable()
36        ht.insert("name", "Faasaaaaaaah")
37        ht.insert("age", 30)
38        print(ht.search("name")) # Output: Alice
39        print(ht.search("age")) # Output: 30
40        print(ht.delete("name")) # Output: Deleted
41        print(ht.search("name")) # Output: Key not found
42        print(ht.delete("name")) # Output: Key not found
43    #c

```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```

Faasaaaaaaah
30
Deleted
Key not found
Key not found
PS C:\Users\Shree priya\Downloads>

```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Generate a simple Python Graph class using an adjacency list with methods to add vertices, add edges, and display the graph with basic docstrings.

class Graph:

"""Simple Graph implementation using adjacency list."""

def __init__(self):

"""Initialize empty graph."""

self.graph = {}

def add_vertex(self, vertex):

"""Add a new vertex to the graph."""

if vertex not in self.graph:

self.graph[vertex] = []

def add_edge(self, v1, v2):

"""Add an edge between two vertices."""

if v1 not in self.graph:

self.add_vertex(v1)

if v2 not in self.graph:

self.add_vertex(v2)

self.graph[v1].append(v2)

self.graph[v2].append(v1)

def display(self):

"""Display the adjacency list of the graph."""

for vertex in self.graph:

```
print(vertex, "->", self.graph[vertex])
```

Example usage:

```
if __name__ == "__main__":
```

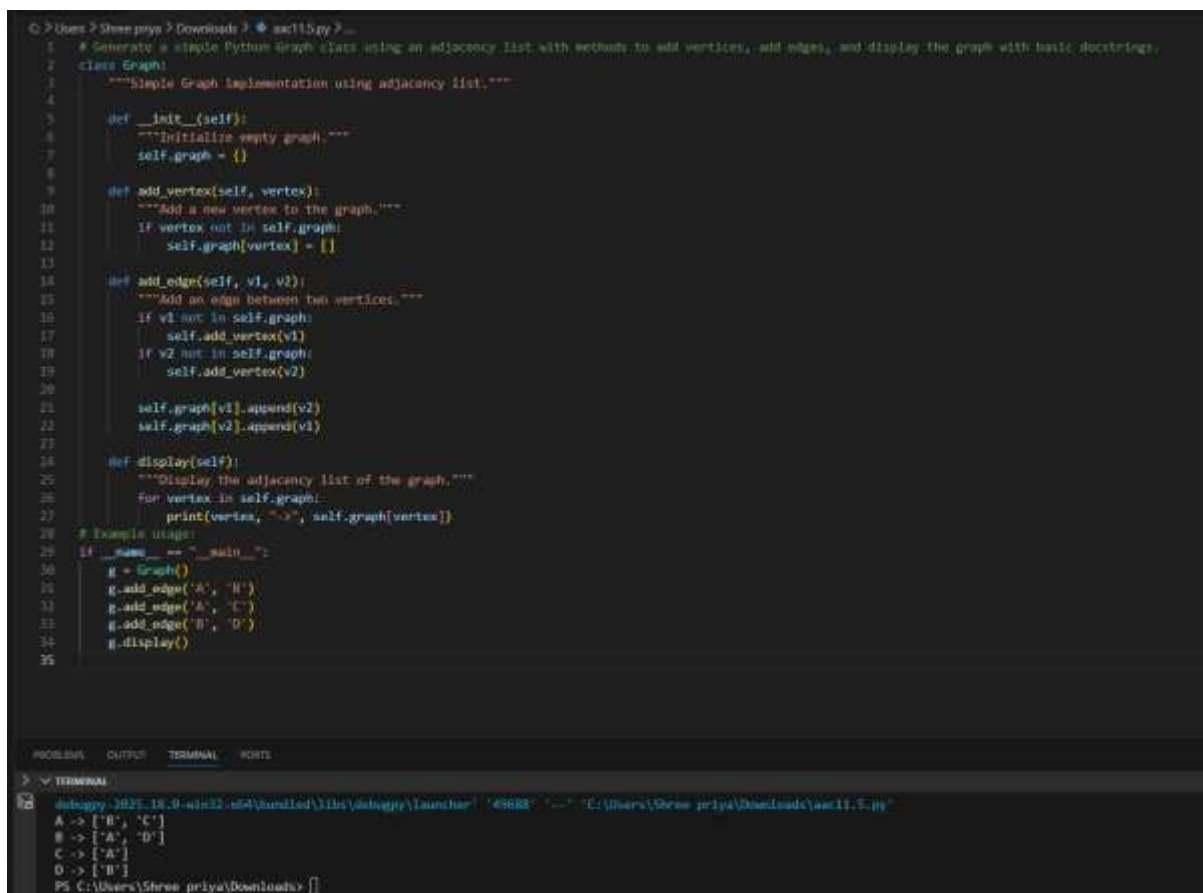
```
    g = Graph()
```

```
    g.add_edge('A', 'B')
```

```
    g.add_edge('A', 'C')
```

```
    g.add_edge('B', 'D')
```

```
    g.display()
```



The screenshot shows a code editor with a Python script for a graph class and its execution in a terminal. The script defines a `Graph` class with methods `__init__`, `add_vertex`, `add_edge`, and `display`. It includes docstrings and an example usage section. The terminal output shows the execution of the script, displaying the adjacency list for each vertex: A -> ['B', 'C'], B -> ['A', 'D'], C -> ['A'], and D -> ['B'].

```
C:\Users\Shree priya> cd Downloads & python3 scil5.py
1 # Generate a simple Python Graph class using an adjacency list with methods to add vertices, add edges, and display the graph with basic docstrings.
2 class Graph:
3     """Simple Graph implementation using adjacency list."""
4
5     def __init__(self):
6         """Initialize empty graph."""
7         self.graph = {}
8
9     def add_vertex(self, vertex):
10        """Add a new vertex to the graph."""
11        if vertex not in self.graph:
12            self.graph[vertex] = []
13
14    def add_edge(self, v1, v2):
15        """Add an edge between two vertices."""
16        if v1 not in self.graph:
17            self.add_vertex(v1)
18        if v2 not in self.graph:
19            self.add_vertex(v2)
20
21        self.graph[v1].append(v2)
22        self.graph[v2].append(v1)
23
24    def display(self):
25        """Display the adjacency list of the graph."""
26        for vertex in self.graph:
27            print(vertex, "->", self.graph[vertex])
28
29 # Example usage:
30 if __name__ == "__main__":
31     g = Graph()
32     g.add_edge('A', 'B')
33     g.add_edge('A', 'C')
34     g.add_edge('B', 'D')
35     g.display()
36
37 PROBLEMS OUTPUT TERMINAL PORTS
38 > TERMINAL
39 debbugy-2021-10-0-win32.exe[sumdled]\lib\debbugy\launcher: '49682' '-' 'E:\Users\Shree priya\Downloads\scil5.py'
40 A -> ['B', 'C']
41 B -> ['A', 'D']
42 C -> ['A']
43 D -> ['B']
44 PS C:\Users\Shree priya\Downloads>
```

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that

handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using

ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack

- o Queue

- o Priority Queue

- o Linked List

- o Binary Search Tree (BST)

- o Graph

- o Hash Table

- o Deque

- Justify your choice in 2–3 sentences per feature.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.

- A functional Python program implementing the chosen feature with comments and docstrings

class PatientQueue:

```
    """Queue implementation for managing patient check-in using FIFO principle."""
```

```
    def __init__(self):
```

```
        """Initialize an empty queue."""
```

```
        self.queue = []
```

```
    def check_in(self, patient):
```

```
        """Add a patient to the queue."""
```

```
self.queue.append(patient)

print(patient, "checked in.")
```

```
def treat_patient(self):
    """Remove and treat the first patient in the queue."""
    if self.is_empty():
        print("No patients waiting.")
    else:
        patient = self.queue.pop(0)
        print(patient, "is being treated.")
```

```
def display_queue(self):
    """Display all waiting patients."""
    if self.is_empty():
        print("No patients in queue.")
    else:
        print("Waiting patients:", self.queue)
```

```
def is_empty(self):
    """Check if queue is empty."""
    return len(self.queue) == 0
```

Sample Usage

```
pq = PatientQueue()
```

```
pq.check_in("Patient A")
```

```
pq.check_in("Patient B")
```

```
pq.check_in("Patient C")
```

```
pq.display_queue()
```

pq.treat_patient()

pq.display_queue()

pq.treat_patient()

```
C:\Users\Shree priya\Downloads> .\mac11.5.py
1  class PatientQueue:
2      self.queue = []
3
4      def check_in(self, patient):
5          """Add a patient to the queue."""
6          self.queue.append(patient)
7          print(patient, "checked in.")
8
9      def treat_patient(self):
10         """Remove and treat the first patient in the queue."""
11         if self.is_empty():
12             print("No patients waiting.")
13         else:
14             patient = self.queue.pop(0)
15             print(patient, "is being treated.")
16
17     def display_queue(self):
18         """Display all waiting patients."""
19         if self.is_empty():
20             print("No patients in queue.")
21         else:
22             print("Waiting patients:", self.queue)
23
24     def is_empty(self):
25         """Check if queue is empty."""
26         return len(self.queue) == 0
27
28 # Sample Usage
29 pq = PatientQueue()
30 pq.check_in("Patient A")
31 pq.check_in("Patient B")
32 pq.check_in("Patient C")
33
34 pq.display_queue()
35
36 pq.treat_patient()
37 pq.display_queue()
38 pq.treat_patient()
```

PROBLEMS OUTPUT TERMINAL PORTS

> TERMINAL

```
debugpy-2025.10.0-win32-x64\debugpy\launcher 62260 -c 'C:\Users\Shree priya\Downloads\mac11.5.py'
Patient A checked in.
Patient B checked in.
Patient C checked in.
Waiting patients: ['Patient A', 'Patient B', 'Patient C']
Patient A is being treated.
Waiting patients: ['Patient B', 'Patient C']
Patient B is being treated.
PS C:\Users\Shree priya\Downloads>
```

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
class TrafficQueue:
```

```
    """Queue implementation for vehicles waiting at a traffic signal."""
```

```
    def __init__(self):
```

```
        """Initialize an empty traffic queue."""
```

```
        self.queue = []
```

```
    def add_vehicle(self, vehicle):
```

```
        """Add a vehicle to the queue."""
```

```
        self.queue.append(vehicle)
```

```
        print(vehicle, "arrived at signal")
```

```
def pass_signal(self):
    """Allow the first vehicle to pass the signal."""
    if self.is_empty():
        print("No vehicles waiting")
    else:
        vehicle = self.queue.pop(0)
        print(vehicle, "passed the signal")
```

```
def display_queue(self):
    """Display vehicles currently waiting."""
    print("Vehicles waiting:", self.queue)
```

```
def is_empty(self):
    """Check if queue is empty."""
    return len(self.queue) == 0
```

Sample Usage

```
tq = TrafficQueue()
```

```
tq.add_vehicle("Car")
```

```
tq.add_vehicle("Bike")
```

```
tq.add_vehicle("Bus")
```

```
tq.display_queue()
```

```
tq.pass_signal()
```

```
tq.display_queue()
```

```
1 class TrafficQueue:
2
3     def __init__(self):
4         """Initialize an empty traffic queue."""
5         self.queue = []
6
7     def add_vehicle(self, vehicle):
8         """Add a vehicle to the queue."""
9         self.queue.append(vehicle)
10        print(vehicle, "arrived at signal")
11
12    def pass_signal(self):
13        """Allow the first vehicle to pass the signal."""
14        if self.is_empty():
15            print("No vehicles waiting")
16        else:
17            vehicle = self.queue.pop(0)
18            print(vehicle, "passed the signal")
19
20    def display_queue(self):
21        """Display vehicles currently waiting."""
22        print("Vehicles waiting:", self.queue)
23
24    def is_empty(self):
25        """Check if queue is empty."""
26        return len(self.queue) == 0
27
28    # Simple Usage
29    tq = TrafficQueue()
30
31    tq.add_vehicle("Car")
32    tq.add_vehicle("Bike")
33    tq.add_vehicle("Bus")
34
35    tq.display_queue()
36
37    tq.pass_signal()
38    tq.display_queue()
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> cd "C:\Users\Shree priya\Downloads"; & "C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe"
debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher "5534" -- "C:\Users\Shree priya\Downloads\src\H1.5.py"
Car arrived at signal
Bike arrived at signal
Bus arrived at signal
Vehicles waiting: ['Car', 'Bike', 'Bus']
Car passed the signal
Vehicles waiting: ['Bike', 'Bus']
PS C:\Users\Shree priya\Downloads>
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

class OrderQueue:

```
    """Queue implementation for processing orders in FIFO order."""
```

```
    def __init__(self):
```

```
        """Initialize an empty order queue."""
```

```
        self.orders = []
```

```
    def place_order(self, order):
```

```
        """Add a new order to the queue."""
```

```
        self.orders.append(order)
```

```
        print(order, "order placed")
```

```
    def process_order(self):
```

```
        """Process the first order in the queue."""
```

```
        if self.is_empty():
```

```
            print("No orders to process")
```

```
        else:
```

```
            order = self.orders.pop(0)
```

```
            print(order, "order processed")
```

```
    def display_orders(self):
```

```
        """Display all pending orders."""
        print("Pending orders:", self.orders)
```

```
def is_empty(self):
    """Check if the queue is empty."""
    return len(self.orders) == 0
```

```
# Sample Usage
```

```
oq = OrderQueue()
```

```
oq.place_order("Order101")
```

```
oq.place_order("Order102")
```

```
oq.place_order("Order103")
```

```
oq.display_orders()
```

```
oq.process_order()
```

```
oq.display_orders()
```

```
1 class OrderQueue:
2     def place_order(self, order):
3         """Add a new order to the queue."""
4         self.orders.append(order)
5         print(order, "order placed")
6
7     def process_order(self):
8         """Process the first order in the queue."""
9         if self.is_empty():
10             print("No orders to process")
11         else:
12             order = self.orders.pop(0)
13             print(order, "order processed")
14
15     def display_orders(self):
16         """Display all pending orders."""
17         print("Pending orders:", self.orders)
18
19     def is_empty(self):
20         """Check if the queue is empty."""
21         return len(self.orders) == 0
22
23 # Sample Usage
24 oq = OrderQueue()
25
26 oq.place_order("Order101")
27 oq.place_order("Order102")
28 oq.place_order("Order103")
29
30 oq.display_orders()
31
32 oq.process_order()
33 oq.display_orders()
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> c::; cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Windows\Common-Programs\Debugpy-launcher' '58209' '...' 'C:\Users\Shree priya\Downloads\aac11.5.py'
Order101 order placed
Order102 order placed
Order103 order placed
Pending orders: ['Order101', 'Order102', 'Order103']
Order101 order processed
Pending orders: ['Order102', 'Order103']
PS C:\Users\Shree priya\Downloads>
```

JUSTIFICATIONS:

Task 1 – Stack Implementation (Justification)

Stack follows ****LIFO (Last In First Out)**** principle.

The last inserted element is removed first.

It is useful for operations like undo, recursion, and expression evaluation.

Task 2 – Queue Implementation (Justification)

Queue follows ****FIFO (First In First Out)**** principle.

Elements are processed in the same order they arrive.

It is commonly used in scheduling and task management systems.

Task 3 – Linked List (Justification)

Linked List stores elements using nodes connected by links.

It allows **dynamic memory allocation**.

Insertion and deletion operations are efficient compared to arrays.

Task 4 – Hash Table (Justification)

Hash Table stores data using **key-value pairs**.

It uses a hash function to quickly locate data.

Searching, insertion, and deletion operations are very fast.

Task 5 – Graph Representation (Justification)

Graph represents relationships between objects.

Nodes represent entities and edges represent connections.

It is useful for networks like roads, social networks, and maps.

Task 6 – Smart Hospital Management System (Justification)

Different hospital operations require different data structures.

Queue manages patient arrival order, while Priority Queue handles emergency cases.

Hash Table, BST, and Graph help manage records, appointments, and navigation efficiently.

Task 7 – Smart City Traffic Control System (Justification)

Traffic systems require organized vehicle and road management.

Queue manages vehicles at signals and Priority Queue handles emergency vehicles.

Hash Table and Graph help manage vehicle data and road network connections.

Task 8 – Smart E-Commerce Platform (Justification)

E-commerce platforms manage orders, products, and delivery routes.

Queue processes orders sequentially while Priority Queue tracks top-selling products.

Hash Table and Graph help in fast product search and delivery route planning.